SLIIT
Discover Your Future

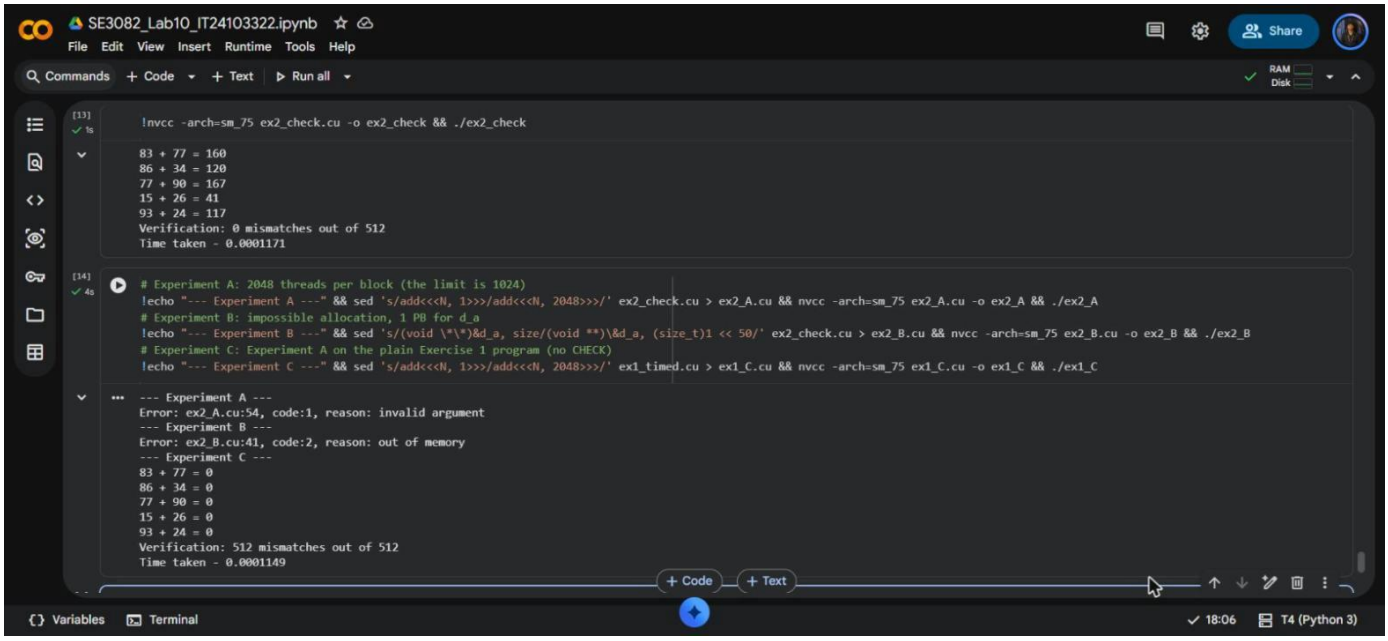
BSc (Hons) in Computer Science
Year 3

Lab Sheets

SE3082 – Parallel Computing **Semster 1, 2026**

IT24103322

Exercise 2



Every cudaMalloc, cudaMemcpy and cudaFree call was wrapped in CHECK(...), and CHECK(cudaGetLastError()) and CHECK(cudaDeviceSynchronize()) were added after the launch. The output is the same as Exercise 1 (0 mismatches out of 512, Time taken - 0.0001171).

Experiment	Change	Output
A – too many threads	add<<<N, 2048>>>	Error: ex2_A.cu:54, code:1, reason: invalid argument
B – impossible allocation	cudaMalloc of (size_t)1 << 50	Error: ex2_B.cu:41, code:2, reason: out of memory
C – no CHECK	Experiment A on Exercise 1	No error; every c is 0, 512 mismatches out of 512

Experiment A was caught by CHECK(cudaGetLastError()) on line 54 (the launch row of the table), because 2048 threads per block is over the limit of 1024. On this CUDA 13.0 runtime the error is reported as code 1, invalid argument, instead of the code 9 shown in the sheet. Experiment B was caught on line 41, the first cudaMalloc (the row for calls that return a value).

In Experiment C the program printed no error at all; it carried on, printed 83 + 77 = 0 and so on, and only the verification loop showed that something was wrong. Without that loop the failure would not be

noticed. The do { ... } while (0) around the macro makes CHECK(x); behave like one normal statement, even inside an if without braces.